

Cost of Queries

Uni Auckland Databases 51

Gerald Weber

Index Types and Metrics

▶ Index Types:

- ▶ **Primary Index** (I_p): An index on a set of attributes that uniquely identify tuples.
- ▶ **Secondary Index** (I_s): An index on non-primary key attributes.
- ▶ **Clustered Index** (I_c): An index where the order of the index entries matches the physical order of the data.
- ▶ **Non-Clustered Index** (I_{nc}): An index where the order of the index entries does not match the physical order of the data.
- ▶ Combine above, eg, **Clustered Primary Index** ($I_{c,p}$): A primary index where the order of the index entries matches the physical order of the data.

▶ Metrics:

- ▶ **Selectivity** (σ): The fraction of tuples that satisfy a condition.
- ▶ **Cardinality** ($|R|$): The number of tuples in a relation R .
- ▶ **Cost** (C): The computational cost of executing a query.

Using Indices in Queries

▶ Example Query:

$$\pi_{ItemName}(\sigma_{Value > 100}(\sigma_{Time = '202504011501'}(Orders) \bowtie OrderItems))$$

▶ Index Notation:

- ▶ $I_{c,s}(Orders.Time)$: Clustered secondary index on Time in Orders.
- ▶ $I_c(OrderItems.OrderID)$: Clustered index on OrderID in OrderItems.

▶ Optimized Query:

$$\pi_{ItemName}(\sigma_{Value > 100}(\sigma_{Time = '202504011501'}(Orders) \bowtie_{I_c} OrderItems))$$

Cost of Index Join in Example

► Scenario:

- Only one order with the timestamp '202504011501'.
- Selectivity: $\sigma_{Value > 100} = 0.30$

► Cost Analysis:

- Retrieve the single matching order from Orders, cost $C_{Order \ \sigma}$.
- Access the index I_c to find matching OrderItems, cost $C_{OrderOrderItems \Join}$.
- **Cost:**

$$C_{query} = C_{Order \ \sigma} + C_{OrderItems \Join}$$

- Minimal cost due to the small number of rows.

► Overall Cost:

- Very low due to the limited data involved, despite selectivity of 0.30.

Cost Analysis with Large Orders and Many Order Items

► Scenario:

- Large number of orders, each with many order items.
- Average number of order items per order: 1,000
- Selectivity ($\sigma_{Value > 100}$): 0.01
- Cost per operation: 0.1 ms

$$|R_{\text{filtered}}| = 0.01 \times 1,000 = 10$$

- Scanning all order items:

$$\text{Cost} = 1,000 \times 0.1 \text{ ms} = 100 \text{ ms}$$

- Processing tuples:

$$\text{cost} = 10 \times 0.1 \text{ ms} = 1 \text{ ms}$$

$$C_{\text{query}} = 100 \text{ ms} + 1 \text{ ms} = 101 \text{ ms}$$

Index on Two Elements

► Definition:

- When an index is created on two elements (composite key), we assume a lexicographic sort on these two columns.

► Example:

- Consider a table `Orders` with columns `CustomerID` and `OrderDate`.
- A clustered index on `(CustomerID, OrderDate)` written as $I_c(\text{CustomerID}, \text{OrderDate})$ will sort the data first by `CustomerID` and then by `OrderDate` within each `CustomerID`.

► Benefits:

- Efficient range queries on the composite key.
- Improved performance for queries that filter by these columns.

► Considerations:

- Only one clustered index can be created per table.
- The choice of columns for the clustered index should be based on the most frequently used queries.

Cost Analysis with Index

- ▶ **Clustered Index:**

- ▶ Clustered index on OrderItems(OrderID, Value):
 $I_c(\text{OrderItems.OrderID}, \text{OrderItems.Value})$

- ▶ **Cost Analysis:**

- ▶ Access the clustered index
 $I_c(\text{OrderItems.OrderID}, \text{OrderItems.Value})$ to find matching OrderItems.
 - ▶ Retrieve the matching orders from Orders.

- ▶ **Overall Cost:**

- ▶ Higher due to the larger data volume and lower selectivity of 0.01.

Comparison:

- ▶ With Index: 2 ms
- ▶ Without Index: 101 ms

Detailed Cost Calculation

► Assumptions:

- Average number of order items per order: 1,000
- Selectivity ($\sigma_{Value > 100}$): 0.01
- Cost per operation: 0.1 ms

► Cardinality:

$$|R_{\text{filtered}}| = 0.01 \times 1,000 = 10$$

► Cost Analysis:

- Accessing index $I_c(\text{OrderItems.OrderID}, \text{OrderItems.Value})$ to find matching OrderItems directly:

$$\text{Cost} = 10 \times 0.1 \text{ ms} = 1 \text{ ms}$$

- Processing tuples:

$$\text{cost} = 10 \times 0.1 \text{ ms} = 1 \text{ ms}$$

► Total Cost:

$$C_{\text{query}} = 1 \text{ ms} + 1 \text{ ms} = 2 \text{ ms}$$

Index as Subscript of Bowtie Operator

- ▶ **Definition:**

- ▶ When an index is used as a subscript (e.g., $\bowtie_{I_c}$), it indicates an **index join**.

- ▶ **Example:**

$$\pi_{ItemName}(\sigma_{Time='202504011501'}(Orders) \bowtie_{I_c} OrderItems)$$

- ▶ Here, I_c is the index on `OrderItems.OrderID`.
- ▶ This notation specifies that the join operation should use the index I_c for efficiency.

Index Join and Its Cost

▶ Index Join:

- ▶ An index join uses an index to quickly locate matching rows in the joined tables.
- ▶ It is particularly efficient when one of the tables has an index on the join key.
- ▶ Example:

$$Orders \bowtie_{I_c} OrderItems$$

Joining Orders and OrderItems using an index on OrderItems.OrderID.

▶ Cost:

▶ Advantages:

- ▶ Reduced I/O operations: Only relevant rows are accessed.
- ▶ Faster join operations: Indexes speed up the retrieval of matching rows.

▶ Disadvantages:

- ▶ Index maintenance: Keeping indexes up-to-date can be costly.
- ▶ Storage overhead: Indexes require additional storage space.

Choosing the Best Indices

- ▶ **Factors to Consider:**

- ▶ **Query Frequency:** How often the query is executed.
- ▶ **Selectivity:** Higher selectivity indices are more beneficial.
- ▶ **Update Cost:** The cost of maintaining the index during insertions, deletions, and updates.

- ▶ **Example:**

- ▶ For frequent queries on Time, a clustered secondary index $I_{c,s}(\text{Orders.Time})$ is beneficial.
- ▶ For queries involving joins on OrderID, a clustered index $I_c(\text{OrderItems.OrderID})$ is crucial.

Example: Primary vs. Secondary Index

- ▶ **Scenario:**

- ▶ We have a query to find items with a value greater than \$100 from orders at a specific time.

- ▶ **Primary Index:**

- ▶ $I_c(\text{OrderItems.OrderID})$: Efficient for joins on OrderID.
 - ▶ Example Query:

$$\pi_{ItemName}(\sigma_{Time='202504011501'}(Orders) \bowtie_{I_c} OrderItems)$$

- ▶ **Performance:**

- ▶ Fast retrieval of orders by OrderID.
 - ▶ Efficient join operation.

- ▶ **Secondary Index:**

- ▶ $I_{c,s}(Orders.Time)$: Efficient for filtering by Time.
 - ▶ **Performance:**
 - ▶ Fast filtering of orders by Time.
 - ▶ Efficient join operation.

Summary

- ▶ Indices significantly improve query performance.
- ▶ Clustered indices are effective for range queries.
- ▶ Selectivity impacts the cost of query execution.
- ▶ High selectivity leads to lower query costs.
- ▶ Proper indexing reduces costs.
- ▶ Index choice should align with query patterns.
- ▶ Efficient indexing strategies are crucial for large datasets.